

Performance Analytics Website with embedded Machine Learning Models

Features

- Complete Analytics and Visualisation of Application Performance using Pandas, Matplotlib, Seaborn (Flask + MongoDB).
- SVM in action to Classify them into Efficient, Moderate or Resource-Intensive using the collected performance metrics data.
- CNN in action to classify application chart images as “memory leak”, “spikes” or “no_issue”.
- Many more interactive features that enables ease navigation to the user.

1. Application Performance Classification

Contents

Search...

Predicted Performance Categories for each Application with Scores

Load details

Test environment details

Observations

Bugs raised

Overall memory usage by each nodetype (gb)

Overall cpu usage by each nodetype (cores)

Contributors to Resource Usage increase/decrease

Complete Analysis piecharts for resource utilizations

Osquery table accuracies

Osquery event accuracies

Disk space usages

Kafka topics

Number of active connections group by application on master

Idle and active postgres connections by applications

Postgres connections by state

Compaction status

Memory usages

Cpu usages

Charts

Pgbadger downloaded report links

View Report

Report generated successfully

Osquery - Multiple customer rule engine and control plane load Performance Report

169_1

 vs

168_2

 vs

167_1

 vs

166_1

 vs

165_1

Predicted Performance Categories for each Application with Scores

Efficient

59

Moderate

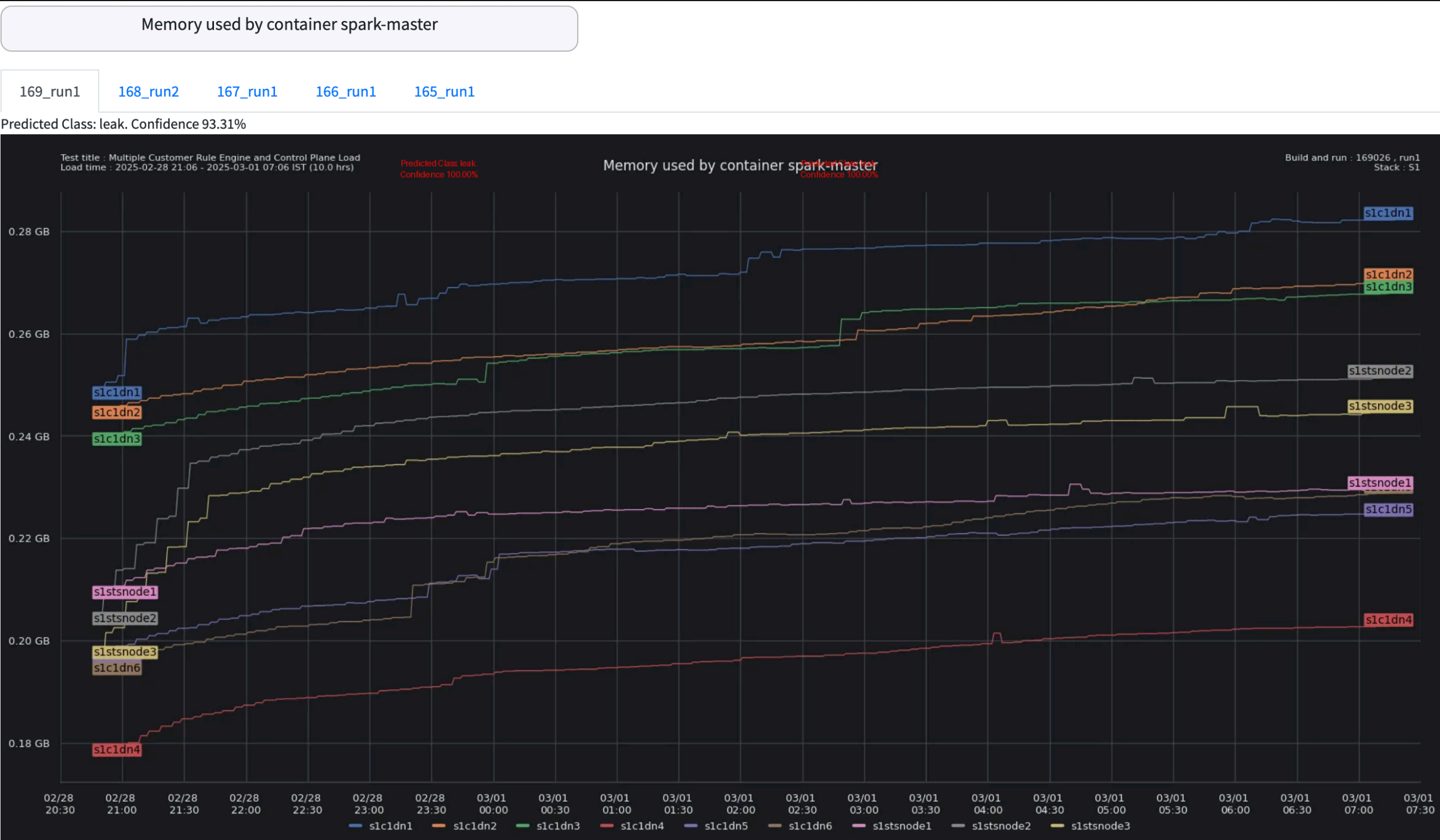
9

Resource-Intensive

3

.*osqLogger.*	Efficient (0.00)
.*redis-server.*	Efficient (0.00)
/app/gprofiler/resources/perf-record--F	Efficient (0.00)
/app/gprofiler/resources/perf-script--F	Efficient (0.00)
/home/airflow/.local/bin/airflow	Efficient (0.00)
/lib/systemd/systemd-udev	Efficient (0.00)
/opt/uptycs/cloud/go/bin/cloudEventDecorator	Efficient (0.00)
/opt/uptycs/cloud/go/bin/cloudEventProcessor	Efficient (0.00)
/opt/uptycs/cloud/go/bin/cloudGraphSynchronizer	Efficient (0.00)
/opt/uptycs/cloud/go/bin/cloudInventoryProcessor	Efficient (0.00)
/opt/uptycs/cloud/go/bin/debeziumConsumer-debeziumconsumer	Efficient (0.00)
/opt/uptycs/cloud/go/bin/decoratorsConsumer	Efficient (0.00)
/opt/uptycs/cloud/go/bin/ruleEngine-production-ruleengine	Resource-Intensive (0.75)
/opt/uptycs/cloud/go/bin/ruleEngine-production-ruleenginecc	Efficient (0.00)
/opt/uptycs/insights/sql/run_pipeline.py	Efficient (0.00)
/opt/uptycs/insights/sql/run_sql.py	Efficient (0.00)
/opt/uptycs/osquery/bin/osqueryd	Efficient (0.00)

2. Resource Chart Classification for identifying Performance Issues

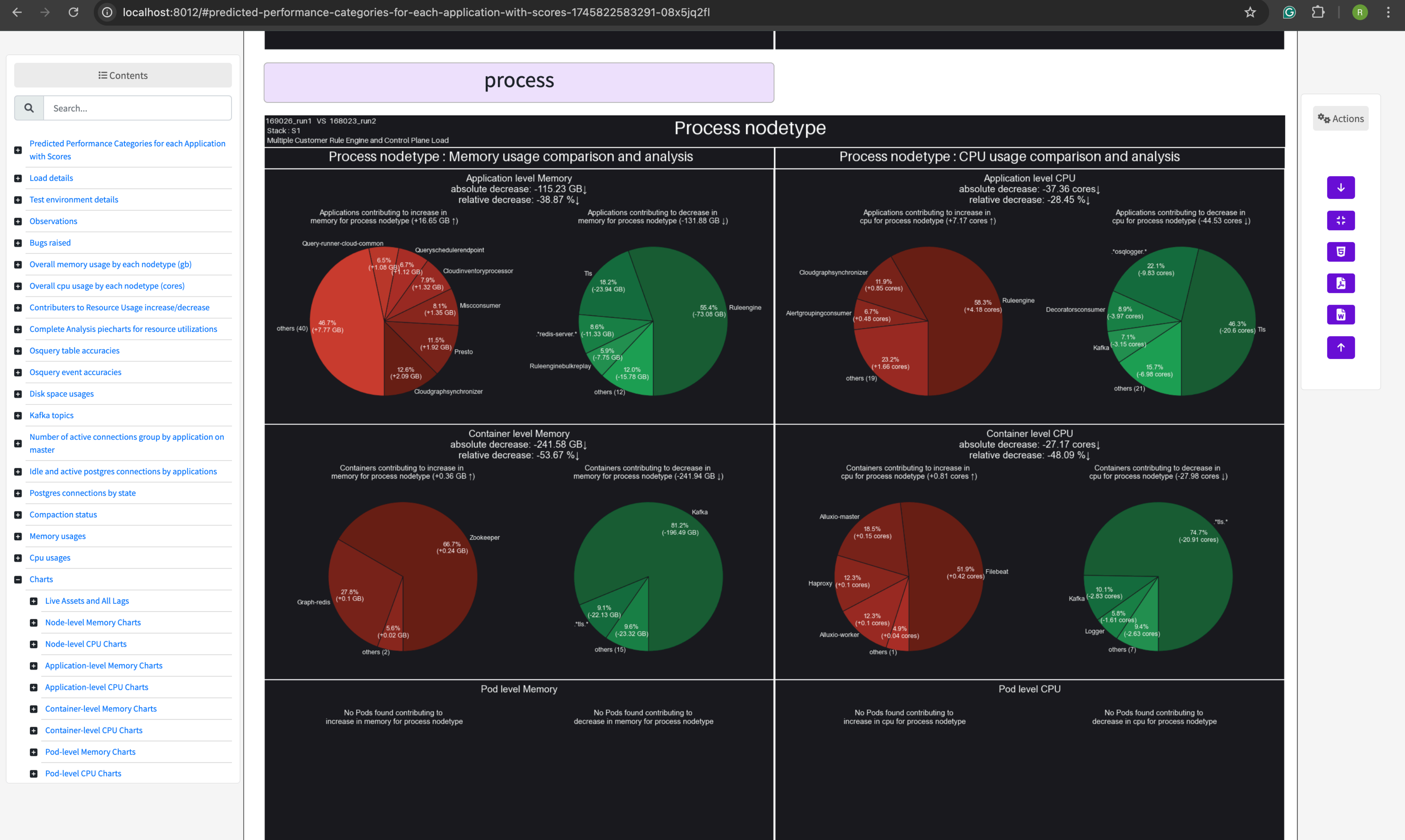


3. More Performance Analytics and Visualisation on UI

3.1. Trend chart showing usages of each node-type over sprints



3.2. Piecharts showing contributions of each application to overall increase/decrease in resource usage for each node-type



4. Other features on the UI

4.1 Home Page

Report Generator

Home

Dashboards

Load Simulator

Resource Dashboards

Detailed comparison and trend analysis of resource utilization over past sprints

Load Simulator

Manage and Run osquery loads

Generate Performance Load Report

Load Type:

Select Load Type

Load Name:

Select Load Name

Select reports to generate/compare

Sprint Field 1 :

Select Sprint

Run Field 1 :

Select Run

Sprint Field 2 :

Select Sprint

Run Field 2 :

Select Run

Confluence URL :

https://uptycsjira.atlassian.net

Email Address :

@uptycs.com

Space :

PERF

API Key :

Enter API key

Parent Page Title :

Enter parent page title

Report Title :

Enter report title

View Report

Publish to Confluence

Logs

\$ report publishing logs will be displayed here

View Report

Report will be displayed here after clicking 'View Report' Button

Contents

Actions

4.2 Applications contributing to increase/decrease in resource usage for each node type.

Contents

Search...

Predicted Performance Categories for each Application with Scores

Load details

Test environment details

Observations

Bugs raised

Overall memory usage by each nodetype (gb)

Overall cpu usage by each nodetype (cores)

Contributors to Resource Usage increase/decrease

Complete Analysis piecharts for resource utilizations

Osquery table accuracies

Osquery event accuracies

Disk space usages

Kafka topics

Number of active connections group by application on master

Idle and active postgres connections by applications

Postgres connections by state

Compaction status

Memory usages

Cpu usages

Charts

Live Assets and All Lags

Node-level Memory Charts

Node-level CPU Charts

Application-level Memory Charts

Application-level CPU Charts

Container-level Memory Charts

Container-level CPU Charts

Pod-level Memory Charts

Pod-level CPU Charts

Contributors to Resource Usage increase/decrease

Memory usages analysis

node_type	Top 5 contributors to increase	Top 5 contributors to decrease
airflow	Airflow_airflow-worker_1 container : 5.77 GB ↑ Airflow-task-supervisor: app : 3.92 GB ↑ Run_pipeline.py app : 0.61 GB ↑ Airflow-task-runner: app : 0.5 GB ↑ Run_sql.py app : 0.35 GB ↑	Uptycs pod : 1.19 GB ↓ Uptycs-osquery pod : 0.64 GB ↓ Airflow_flower_1 container : 0.34 GB ↓ Moby app : 0.29 GB ↓ Airflow app : 0.21 GB ↓
cloudquery	Cloudquery-jupiter1-aws-624 container : 0.22 GB ↑ Cloudquery-jupiter1-gcp-776 container : 0.04 GB ↑ 104 container : 0.04 GB ↑ 464 container : 0.04 GB ↑ Container app : 0.03 GB ↑	Osqueryd app : 0.01 GB ↓ Filebeat container : 0.01 GB ↓ others (0) app : 0.0 GB ↓ others (0) container : 0.0 GB ↓
data	Data-archival app : 26.84 GB ↑ Spark-worker container : 13.61 GB ↑ Orc-compaction app : 6.71 GB ↑ Genericstatemanagerexecutor app : 4.09 GB ↑ others (7) app : 3.19 GB ↑	Hdfs-dn container : 283.97 GB ↓ Presto container : 230.55 GB ↓ Trino app : 230.21 GB ↓ Osqueryingestion app : 35.13 GB ↓ others (6) container : 27.59 GB ↓
ep	Kubelet.conf app : 0.04 GB ↑ Software_update=false app : 0.04 GB ↑ Spark-worker app : 0.03 GB ↑ app : 0.01 GB ↑ Init-maybe-ubiquity app : 0.01 GB ↑	Effective-permissions container : 1.5 GB ↓ Effectivepermissions app : 1.11 GB ↓ Sts-detection container : 0.9 GB ↓ Granulate-gprofiler container : 0.64 GB ↓ Stsdetection app : 0.64 GB ↓
pg	Postgres-statedb15 container : 21.22 GB ↑ Uptycs-metastoredb app : 19.58 GB ↑ Postgres-rangerdb15 container : 16.58 GB ↑ Postgres-configdb15 container : 11.14 GB ↑ Postgres-metastoredb15 container : 8.42 GB ↑	Uptycs-configdb app : 16.92 GB ↓ Startup-recovering app : 7.88 GB ↓ Checkpointner app : 3.56 GB ↓ Postgres-kube-vaultdb15 container : 0.61 GB ↓ others (7) app : 0.47 GB ↓
process	others (40) app : 7.77 GB ↑ Cloudgraphsynchronizer app : 2.09 GB ↑ Presto app : 1.92 GB ↑ Miscconsumer app : 1.35 GB ↑ Cloudinventoryprocessor app : 1.32 GB ↑	Kafka container : 196.49 GB ↓ Ruleengine app : 73.08 GB ↓ Tls app : 23.94 GB ↓ others (15) container : 23.32 GB ↓ *.tls.* container : 22.13 GB ↓

Actions

↓

↕

📄

📄

📄

↑

Cpu usage analysis

node_type	Top 5 contributors to increase	Top 5 contributors to decrease
airflow	Airflow_airflow-worker_1 container : 5.09 cores ↑ Airflow-task-supervisor: app : 3.63 cores ↑ others (16) app : 1.31 cores ↑ Run_pipeline.py app : 1.27 cores ↑ Run_sql.py app : 0.89 cores ↑	Uptycs pod : 0.27 cores ↓ Airflow_airflow-scheduler_1 container : 0.12 cores ↓ Airflow app : 0.08 cores ↓ others (0) app : 0.0 cores ↓ others (0) container : 0.0 cores ↓

⚙️ Update Simulator Parameters

🚀 Load Type:

Osquery

📁 Stack:

s1_nodes.json

👤 Number of Customers to send load to:

100

👤 Total Number of Assets to Simulate:

10000

📊 First X% of customers:

Select X = 50

👤 Gets Y% of total load:

Select Y = 50

🕒 No.of msgs (msgs x 4 = Load Duration in seconds):

9000

📄 Input File:

Select Input File

⌵ ⚙️ Update Params

👁️ View Asset Distribution

📄 More about inputfile

Note: Asset distribution will be done only on the selected simulators. So please validate the selected simulators before proceeding.

● 1 / 2 Simulators Online

✓ 2 / 2 Simulators Selected

🔄 0 Total Live assets in configdb

✓ Select all

🔄 Pull latest code

CPU / Mem

🔄 Refresh

💀 Kill Instances

👤+ Enroll Assets

▶ Start Load

■ Stop Load

🔄 Online ✓

🖥️ localhost

🔄 0 Live assets in configdb

0

live instances

100

exp. instances

10000

assets to enroll

227

msgs to send

00:15:08

load duration

🔄 Offline ✓

🖥️ machine2

🔄 0 Live assets in configdb

Failed to connect to the simulator server
'http://machine2:8123/check_sim_health':
HTTPConnectionPool(host='machine2', port=8123): Max retries
exceeded with url: /check_sim_health (Caused by
NameResolutionError(": Failed to resolve 'machine2' ([Errno 8]
nodename nor servname provided, or not known"))

▼🔧 Update Params

👁 View Asset Distribution

📄 More about inputfile

Note: Asset distribution will be done only on the selected simulators. So please validate the selected simulators before proceeding.

📊 Asset Distribution Logic

Key	Value																					
1.Total assets to enroll	10000																					
2.Total number of customers	100																					
3. Number of simulators configured	2																					
4.Asset Distribution for each customer	<table><tr><th>index</th><th>assets</th></tr><tr><td>0</td><td>100</td></tr><tr><td>1</td><td>100</td></tr><tr><td>2</td><td>100</td></tr><tr><td>3</td><td>100</td></tr><tr><td>4</td><td>100</td></tr><tr><td>5</td><td>100</td></tr></table>	index	assets	0	100	1	100	2	100	3	100	4	100	5	100							
index	assets																					
0	100																					
1	100																					
2	100																					
3	100																					
4	100																					
5	100																					
5.First 50.0% (50) of customers gets	5000 assets.																					
6.And The last 50.0% (50) of customers gets	5000 assets.																					
simulator 'localhost' instance list: (assets=5000)	<table><tr><th>index</th><th>domain</th><th>assets</th></tr><tr><td>0</td><td>jupiter</td><td>100</td></tr><tr><td>1</td><td>jupiter2</td><td>100</td></tr><tr><td>2</td><td>jupiter4</td><td>100</td></tr><tr><td>3</td><td>jupiter6</td><td>100</td></tr><tr><td>4</td><td>jupiter8</td><td>100</td></tr><tr><td>5</td><td>jupiter10</td><td>100</td></tr></table>	index	domain	assets	0	jupiter	100	1	jupiter2	100	2	jupiter4	100	3	jupiter6	100	4	jupiter8	100	5	jupiter10	100
index	domain	assets																				
0	jupiter	100																				
1	jupiter2	100																				
2	jupiter4	100																				
3	jupiter6	100																				
4	jupiter8	100																				
5	jupiter10	100																				
simulator 'machine2' instance list: (assets=5000)	<table><tr><th>index</th><th>domain</th><th>assets</th></tr><tr><td>0</td><td>jupiter1</td><td>100</td></tr><tr><td>1</td><td>jupiter3</td><td>100</td></tr><tr><td>2</td><td>jupiter5</td><td>100</td></tr><tr><td>3</td><td>jupiter7</td><td>100</td></tr><tr><td>4</td><td>jupiter9</td><td>100</td></tr><tr><td>5</td><td>jupiter11</td><td>100</td></tr></table>	index	domain	assets	0	jupiter1	100	1	jupiter3	100	2	jupiter5	100	3	jupiter7	100	4	jupiter9	100	5	jupiter11	100
index	domain	assets																				
0	jupiter1	100																				
1	jupiter3	100																				
2	jupiter5	100																				
3	jupiter7	100																				
4	jupiter9	100																				
5	jupiter11	100																				

Asset distribution logic calculated. ✕